

Dokumentacja

Dashboard do wizualizacji danych SQL z Archiwum

Miejsce Narodowe Centrum Promieniowania Synchrotronowego SOLARIS
Autor: Karol Ożóg
Data: Wrzesień 2026

Aplikacja webowa umożliwiająca wizualizację danych pomiarowych przechowywanych w bazach SQL Server. Aplikacja została wykonana w języku Python z wykorzystaniem frameworka Streamlit oraz bibliotek **pandas**, **pyodbc**, **plotly** i **numpy**. Dodatkowo projekt zawiera skrypt **start.vbs**, który automatyzuje uruchamianie, wykrywanie działającej instancji oraz bezpieczny restart aplikacji w systemie Windows.

Kraków, 2026

Spis treści

1	Przygotowanie i uruchomienie aplikacji	1
1.1	Wymagania systemowe	1
1.2	Struktura plików projektu	1
1.3	Przygotowanie Pythona na komputerze z dostępem do Internetu	1
1.4	Przeniesienie plików na komputer docelowy	2
1.5	Utworzenie środowiska wirtualnego	2
1.6	Instalacja bibliotek bez dostępu do Internetu	2
1.7	Uruchomienie aplikacji z terminala	2
1.8	Uruchamianie za pomocą <code>start.vbs</code>	3
1.9	Sprawdzanie interpretera i pliku aplikacji	3
1.10	Wykrywanie adresu IPv4	3
1.11	Plik <code>link.txt</code>	4
1.12	Sprawdzanie portu 8501	4
1.13	Bezpieczny restart aplikacji	4
1.14	Odczytywanie szczegółów procesu	4
1.15	Weryfikacja ścieżki interpretera	5
1.16	Weryfikacja pliku <code>app.py</code>	5
1.17	Zakończenie i ponowne uruchomienie procesu	5
2	Opis działania aplikacji	5
2.1	Punkt wejścia aplikacji – <code>app.py</code>	5
2.2	Konfiguracja strony Streamlit	6
2.3	Pobieranie listy baz danych	6
2.4	Wybór bazy danych	6
2.5	Reset serii po zmianie bazy	6
2.6	Pobieranie tabel	7
2.7	Pobieranie nazw opisowych zmiennych	7
3	Konfiguracja serii danych	7
3.1	Lista dostępnych tabel	7
3.2	Panel serii danych	7
3.3	Identyfikatory serii	8
3.4	Wybór zmiennej	8
3.5	Ręczny zakres osi Y dla serii	8
3.6	Dodawanie i usuwanie serii	9
4	Moduł <code>config.py</code>	9
4.1	Strefa czasowa	9
4.2	Limity i parametry wykresu	9
4.3	Cache Streamlit	10
4.4	Konfiguracja połączenia z bazą	10
4.5	Paleta kolorów	10
5	Moduł <code>db.py</code>	10
5.1	Pooling połączeń	11
5.2	Tworzenie połączenia	11
5.3	Cytowanie nazw schematów i tabel	11
5.4	Pobieranie listy baz danych	11
5.5	Pobieranie listy tabel	12
5.6	Pobieranie listy zmiennych	12
5.7	Mapowanie identyfikatora na nazwę	13
5.8	Najważniejsze zapytanie – <code>load_data</code>	13
5.8.1	Etap pierwszy – CTE <code>filtered</code>	14
5.8.2	Etap drugi – CTE <code>bounds</code>	14
5.8.3	Etap trzeci – CTE <code>bucketed</code>	15
5.8.4	Etap czwarty – CTE <code>aggregated</code>	15
5.8.5	Wyznaczenie czasu reprezentującego bucket	16
5.8.6	Sortowanie wyniku SQL	16

5.9	Konwersja danych w Pandas	16
6	Moduł state.py	17
6.1	Stan sesji Streamlit	17
6.2	Domyślna seria	17
6.3	Domyślny zakres czasu	18
6.4	Tryby wyboru zakresu	18
6.5	Szybkie zakresy czasu	18
6.6	Tryb ostatniego zakresu ciągłego	18
6.7	Własny zakres daty i godziny	19
6.8	Funkcja <code>insert_time</code>	19
6.9	Dodawanie serii	19
6.10	Usuwanie serii	20
6.11	Resetowanie serii	20
6.12	Mechanizm box-select i zoomu	20
6.13	Cofnięcie zoomu	21
7	Moduł charting.py	21
7.1	Unikalne klucze wykresu	21
7.2	Konwersja czasu	22
7.3	Konfiguracja osi Y	22
7.4	Rozmieszczanie osi Y	22
7.5	Dodawanie wykresów Plotly	23
7.6	Budowa figury	24
7.7	Wartości MIN i MAX	24
7.8	Układ wykresu	24
7.9	Obsługa zdarzeń Plotly	25
7.9.1	Odczyt atrybutów zdarzenia	25
7.9.2	Punkty i zaznaczenia box-select	25
7.9.3	Rozpoznawanie serii na podstawie <code>customdata</code>	25
7.10	Markery M1 i M2	26
7.10.1	Włączenie trybu markerów	26
7.10.2	Odczyt klikniętego punktu	26
7.10.3	Zapisywanie dwóch markerów	26
7.10.4	Wizualizacja markerów	27
7.10.5	Informacje o markerach	27
7.10.6	Czyszczenie markerów	27
8	Fragment Streamlit i automatyczne odświeżanie	27
8.1	Funkcja <code>make_chart_fragment</code>	27
8.2	Ustawienia odświeżania	28
8.3	Ręczne odświeżenie	28
9	Update	28
9.1	Wielopoziomowe cofanie zoomu (<code>state.py</code>)	28
9.2	Prawy przycisk myszy = box select, lewy = zoom (<code>charting</code>)	28
9.3	Naprawa znikających linii MIN/MAX po cofnięciu	29

1 Przygotowanie i uruchomienie aplikacji

Poniżej opisano przygotowanie i uruchomienie aplikacji napisanej w języku Python z wykorzystaniem biblioteki Streamlit. Aplikacja służy do wizualizacji historycznych danych pomiarowych zapisanych w bazach Microsoft SQL Server. Aplikacja zakłada pracę w środowisku Windows, ponieważ korzysta z:

- uwierzytelniania Windows poprzez `Trusted_Connection=yes`,
- sterownika ODBC dla Microsoft SQL Server,
- skryptu uruchomieniowego napisanego w języku VBScript,
- poleceń systemowych takich jak `netstat`, `tasklist`, `taskkill`, `ipconfig` oraz PowerShell.

1.1 Wymagania systemowe

Do uruchomienia aplikacji wymagane są:

- Python w wersji 3.10 lub nowszej,
- dostęp do serwera Microsoft SQL Server,
- zainstalowany sterownik ODBC Driver 17 for SQL Server,
- możliwość używania uwierzytelniania Windows,
- system Windows w przypadku korzystania z `restart.vbs`,
- uprawnienia pozwalające na uruchamianie procesów Python i odczytywanie informacji o procesach systemowych.

Aplikacja wykorzystuje następujące biblioteki Python:

- `streamlit` – budowa interfejsu webowego,
- `pandas` – odczyt, konwersja i obróbka danych,
- `plotly` – tworzenie interaktywnych wykresów,
- `pyodbc` – komunikacja z SQL Server,
- `numpy` – interpolacja wartości dla wybranych punktów.

1.2 Struktura plików projektu

Aktualna wersja projektu została podzielona na moduły odpowiadające poszczególnym zadaniom aplikacji. Główna logika interfejsu znajduje się w pliku `app.py`, natomiast obsługa danych, wykresów i stanu sesji została przeniesiona do osobnych plików.

Główne pliki projektu to:

- `app.py` – główny punkt wejścia aplikacji Streamlit,
- `db.py` – połączenie z SQL Serverem, zapytania SQL, pobieranie i agregacja danych,
- `charting.py` – tworzenie wykresów Plotly, obsługa osi, markerów, tabel podsumowania i zdarzeń wykresu,
- `state.py` – zarządzanie stanem sesji, seriami, zakresem czasu oraz zoomem,
- `config.py` – stałe konfiguracyjne, parametry bazy, limity aplikacji i kolory serii,
- `start.vbs` – uruchamianie oraz bezpieczny restart aplikacji w systemie Windows,
- `README.md` – skrócona dokumentacja projektu.

1.3 Przygotowanie Pythona na komputerze z dostępem do Internetu

Na komputerze posiadającym dostęp do Internetu należy przygotować interpreter Python w wersji zgodnej z wymaganiami aplikacji.

Jeżeli aplikacja będzie później uruchamiana bez dostępu do Internetu, należy wcześniej pobrać lokalne kopie wszystkich wymaganych pakietów.

W katalogu projektu należy utworzyć katalog:

```
packages
```

Następnie należy wykonać polecenie:

```
py -m pip download -d .\packages streamlit pandas plotly pyodbc numpy
```

Polecenie `pip download` nie instaluje bibliotek w aktualnym Pythonie. Pobiera ono pliki instalacyjne pakietów oraz ich zależności do wskazanego katalogu.

Jest to istotne w przypadku komputera docelowego, który nie posiada dostępu do Internetu. W takiej sytuacji instalacja może później zostać wykonana wyłącznie z lokalnego katalogu `packages`.

1.4 Przeniesienie plików na komputer docelowy

Na komputer docelowy należy przenieść:

- interpreter Python lub jego instalator,
- katalog projektu,
- pliki `app.py`, `db.py`, `charting.py`, `state.py` i `config.py`,
- plik `start.vbs`,
- katalog `packages` zawierający pakiety Python.

W przypadku korzystania ze skryptu `start.vbs` szczególne znaczenie mają ścieżki do interpreterów Python. Skrypt oczekuje między innymi następujących lokalizacji:

```
.venv\Scripts\python.exe  
python314\python.exe
```

Do uruchomienia aplikacji skrypt wykorzystuje przede wszystkim:

```
.venv\Scripts\python.exe
```

Drugi interpreter, znajdujący się w katalogu `python314`, jest używany podczas sprawdzania, czy proces zajmujący port 8501 należy do jednej z akceptowanych instancji Pythona.

1.5 Utworzenie środowiska wirtualnego

Na komputerze docelowym należy utworzyć środowisko wirtualne przy użyciu lokalnego interpretera Python.

Przykładowe polecenie:

```
.\Python\python.exe -m venv .venv
```

Jeżeli interpreter znajduje się bezpośrednio w systemie, można użyć:

```
python -m venv .venv
```

Po wykonaniu polecenia w katalogu projektu powstanie katalog:

```
.venv
```

Środowisko wirtualne zawiera niezależny interpreter oraz własny katalog pakietów. Dzięki temu biblioteki używane przez aplikację nie muszą być instalowane globalnie w systemie.

1.6 Instalacja bibliotek bez dostępu do Internetu

Jeżeli katalog `packages` został wcześniej przygotowany na komputerze z dostępem do Internetu, biblioteki można zainstalować lokalnie:

```
.\.venv\Scripts\python.exe -m pip install --no-index --find-links=.packages streamlit pandas plotly pyodbc numpy
```

Parametr `-no-index` powoduje, że `pip` nie próbuje korzystać z internetowego repozytorium PyPI.

Parametr `-find-links=.` wskazuje katalog zawierający lokalne pliki instalacyjne.

1.7 Uruchomienie aplikacji z terminala

Aplikację można uruchomić z aktywnego środowiska wirtualnego:

```
streamlit run app.py
```

Można również użyć interpretera znajdującego się bezpośrednio w środowisku:

```
.\.venv\Scripts\python.exe -m streamlit run .\app.py
```

Aplikacja domyślnie uruchamia serwer Streamlit na porcie:

```
8501
```

Po uruchomieniu powinna być dostępna pod adresem:

```
http://localhost:8501
```

Aktualny kod aplikacji nie przekazuje parametrów `-server.address` ani `-server.port`. Oznacza to, że używane są domyślne ustawienia Streamlit, w tym domyślny port 8501.

1.8 Uruchamianie za pomocą start.vbs

Plik `start.vbs` jest skryptem Windows Script Host napisanym w języku VBScript. Jego zadaniem jest uruchomienie aplikacji bez wyświetlania okna CMD lub PowerShell.

Skrypt:

- ustala katalog, w którym znajduje się plik `start.vbs`,
- sprawdza istnienie interpretera Python,
- sprawdza istnienie pliku `app.py`,
- wyszukuje adres IPv4 komputera,
- sprawdza, czy port 8501 jest zajęty,
- uruchamia aplikację, jeśli port jest wolny,
- otwiera przeglądarkę pod adresem lokalnym,
- w przypadku działającej aplikacji sprawdza, czy można ją bezpiecznie zakończyć i uruchomić ponownie.

Skrypt używa następujących ścieżek względnych:

```
.venv\Scripts\python.exe
python314\python.exe
app.py
link.txt
```

Ścieżki są budowane względem katalogu, w którym znajduje się skrypt. Dzięki temu nie jest wymagane uruchamianie go z konkretnego katalogu roboczego.

1.9 Sprawdzanie interpretera i pliku aplikacji

Na początku skrypt sprawdza, czy istnieje interpreter:

```
.venv\Scripts\python.exe
```

Jeżeli plik nie istnieje, wyświetlany jest komunikat:

```
Brak Pythona:
ścieżka_do_.venv\Scripts\python.exe
```

Następnie sprawdzane jest istnienie głównego pliku aplikacji:

```
app.py
```

Jeżeli plik nie istnieje, skrypt nie uruchamia Streamlit i wyświetla odpowiedni komunikat błędu.

1.10 Wykrywanie adresu IPv4

Skrypt wykonuje polecenie:

```
ipconfig
```

Wynik polecenia jest tymczasowo zapisywany do pliku:

```
ipconfig_temp.txt
```

Następnie skrypt wyszukuje linię zawierającą tekst IPv4. Jeżeli znaleziony adres nie zaczyna się od 127, jest traktowany jako adres sieciowy komputera.

Na tej podstawie budowany jest adres:

```
http://ADRES_IP:8501
```

Jeżeli adres IPv4 nie zostanie znaleziony, do pliku `link.txt` zostaje wpisana informacja:

```
Nie znaleziono adresu IPv4
```

Po zakończeniu odczytu plik tymczasowy `ipconfig_temp.txt` jest usuwany.

1.11 Plik link.txt

Podczas każdego uruchomienia skrypt tworzy lub nadpisuje plik:

```
link.txt
```

Plik zawiera:

```
WIZUALIZACJA BAZY DANYCH
=====
```

```
Local URL:
http://localhost:8501
```

```
Network URL:
http://ADRES_IP:8501
```

Plik pozwala szybko sprawdzić adres lokalny oraz adres dostępny dla innych komputerów w tej samej sieci, o ile konfiguracja systemu i zapora sieciowa na to pozwalają.

1.12 Sprawdzanie portu 8501

Przed uruchomieniem aplikacji skrypt sprawdza, czy port 8501 jest zajęty:

```
netstat -ano | findstr LISTENING | findstr :8501
```

Jeżeli nie zostanie znaleziony żaden proces nasłuchujący na porcie, skrypt uruchamia aplikację:

```
".venv\Scripts\python.exe" -m streamlit run "app.py"
```

Po odczekaniu dwóch sekund otwierany jest adres:

```
http://localhost:8501
```

1.13 Bezpieczny restart aplikacji

Jeżeli port 8501 jest już zajęty, skrypt nie kończy automatycznie znalezionej aplikacji. Najpierw sprawdza, jaki proces korzysta z portu oraz czy jest to instancja aplikacji.

W pierwszym kroku pobierany jest identyfikator procesu PID. Następnie wykonywane jest polecenie:

```
tasklist /FI "PID eq NUMER_PID" /FO CSV /NH
```

Skrypt sprawdza, czy obraz procesu ma nazwę:

```
python.exe
```

lub:

```
pythonw.exe
```

Jeżeli port zajmuje inny program, skrypt wyświetla ostrzeżenie i nie zabija procesu.

1.14 Odczytywanie szczegółów procesu

Jeżeli proces na porcie 8501 jest procesem Python, skrypt próbuje odczytać:

- pełną ścieżkę pliku wykonywalnego interpretera,
- pełną linię polecenia używaną do uruchomienia procesu.

W pierwszej kolejności używany jest PowerShell oraz polecenie `Get-CimInstance` dla klasy `Win32_Process`.

Jeżeli metoda PowerShell nie zadziała, skrypt korzysta z zapasowej metody:

```
wmic process where ProcessId=PID get ExecutablePath,CommandLine /VALUE
```

Odczytanie `ExecutablePath` i `CommandLine` pozwala upewnić się, że proces jest rzeczywiście procesem należącym do tej aplikacji.

1.15 Weryfikacja ścieżki interpretera

Skrypt normalizuje ścieżki, między innymi:

- usuwa cudzysłowy,
- zamienia ukośniki na odwrotne ukośniki,
- usuwa zbędne spacje,
- ignoruje wielkość liter.

Następnie porównuje ścieżkę interpretera działającego procesu z dwiema akceptowanymi ścieżkami:

```
.venv\Scripts\python.exe  
python314\python.exe
```

Jeżeli ścieżka procesu nie odpowiada żadnej z nich, proces nie jest zabijany.

1.16 Weryfikacja pliku app.py

Oprócz interpretera sprawdzana jest również linia polecenia procesu. Musi ona zawierać oczekiwaną ścieżkę do pliku:

```
app.py
```

Jeżeli proces Python został uruchomiony z innym plikiem lub nie udało się odczytać jego linii polecenia, skrypt nie wykonuje restartu.

Dopiero gdy:

1. port jest zajęty przez Python,
2. ścieżka interpretera jest zgodna,
3. linia polecenia zawiera właściwy plik `app.py`,

skrypt uznaje proces za własną instancję aplikacji.

1.17 Zakończenie i ponowne uruchomienie procesu

Po pozytywnej weryfikacji wykonywane jest polecenie:

```
taskkill /PID NUMER_PID /T /F
```

Parametr `/PID` wskazuje proces, który ma zostać zakończony.

Parametr `/T` powoduje zakończenie również procesów potomnych.

Parametr `/F` wymusza zakończenie procesu.

Po wykonaniu polecenia skrypt ponownie sprawdza, czy proces zniknął z systemu. Jeżeli nadal działa, nowa instancja aplikacji nie jest uruchamiana.

Jeżeli proces został poprawnie zakończony, skrypt uruchamia nową instancję Streamlit i otwiera stronę lokalną w przeglądarce.

2 Opis działania aplikacji

Aplikacja służy do odczytu, agregowania i wizualizacji danych pomiarowych z baz Microsoft SQL Server.

2.1 Punkt wejścia aplikacji – app.py

Plik `app.py` jest głównym punktem wejścia aplikacji. To właśnie w tym pliku wykonywany jest cały przepływ działania interfejsu:

1. konfiguracja strony Streamlit,
2. inicjalizacja stanu sesji,
3. pobranie listy baz danych,
4. wybór bazy,
5. pobranie tabel,
6. pobranie mapowania nazw zmiennych,
7. konfiguracja serii,
8. wybór zakresu czasu,
9. konfiguracja odświeżania,
10. pobranie danych,
11. przypisanie kolorów,
12. utworzenie i wyświetlenie wykresu,
13. wyświetlenie tabel danych zagregowanych.

2.2 Konfiguracja strony Streamlit

Konfiguracja strony jest wykonywana przed pozostałymi elementami interfejsu:

```
st.set_page_config(
    page_title="Wizualizacja bazy danych",
    page_icon="",
    layout="wide",
)
```

Parametr `page_title` ustala tytuł karty przeglądarki.

Parametr `page_icon` ustala ikonę aplikacji.

Parametr `layout="wide"` powoduje wykorzystanie szerokiego układu, co jest szczególnie istotne w przypadku wykresu zawierającego kilka osi Y.

Po konfiguracji strony wywoływana jest funkcja:

```
init_session_state()
```

Po uruchomieniu aplikacja wyświetla tytuł, oraz opis.

Następnie rozpoczyna się przygotowanie panelu bocznego oraz pobieranie danych z serwera SQL Server.

2.3 Pobieranie listy baz danych

Lista baz jest pobierana za pomocą funkcji:

```
available_databases = load_databases()
```

Jeżeli wystąpi wyjątek podczas komunikacji z serwerem, użytkownik otrzymuje komunikat:

Nie udało się pobrać listy baz danych z serwera.

Dodatkowo wyświetlany jest tekst wyjątku za pomocą `st.code`. Następnie wykonywane jest `st.stop()`, które zatrzymuje dalsze tworzenie interfejsu.

Jeżeli lista baz jest pusta, wyświetlane jest ostrzeżenie:

Na serwerze nie znaleziono żadnych baz danych.

2.4 Wybór bazy danych

Baza danych wybierana jest w panelu bocznym za pomocą:

```
st.sidebar.selectbox(
    "Baza danych",
    available_databases,
    key="selected_database",
)
```

Domyślnie preferowana jest baza określona w:

```
DEFAULT_DB_NAME = "Archives"
```

Jeżeli baza **Archives** znajduje się na liście dostępnych baz, zostaje wybrana automatycznie. W przeciwnym przypadku aplikacja wybiera pierwszą bazę z listy.

2.5 Reset serii po zmianie bazy

Aplikacja zapamiętuje poprzednio wybraną bazę w kluczu:

```
_last_database
```

Po zmianie bazy wywoływana jest funkcja:

```
reset_series()
```

Reset jest konieczny, ponieważ serie skonfigurowane dla poprzedniej bazy mogą wskazywać na tabele lub zmienne, które nie istnieją w nowej bazie.

Funkcja pozostawia jedną pustą serię, aby użytkownik mógł od razu wybrać nową tabelę i zmienną.

2.6 Pobieranie tabel

Po wybraniu bazy aplikacja wykonuje:

```
tables_df = load_tables(selected_database)
```

Wynik jest przechowywany w obiekcie typu `pandas.DataFrame`.

Jeżeli połączenie z bazą nie powiedzie się, wyświetlany jest komunikat:

Nie udało się połączyć z bazą SQL Server.

Jeżeli baza nie zawiera żadnych tabel, aplikacja wyświetla ostrzeżenie i zatrzymuje dalsze działanie.

2.7 Pobieranie nazw opisowych zmiennych

Aplikacja próbuje pobrać mapowanie identyfikatorów zmiennych na nazwy opisowe:

```
variable_names = load_variable_names(selected_database)
```

Mapowanie jest pobierane z tabeli określonej w `config.py`:

```
NAZWY_TABLE = "dbo.WODA_VARIABLES"
```

Jeżeli tabela nie istnieje lub zapytanie zakończy się błędem, aplikacja nie zatrzymuje całego interfejsu. Zamiast tego ustawia pusty słownik:

```
variable_names = {}
```

Użytkownik otrzymuje ostrzeżenie w panelu bocznym, a zmienne są wyświetlane z wykorzystaniem samych identyfikatorów.

3 Konfiguracja serii danych

3.1 Lista dostępnych tabel

Tabele są prezentowane użytkownikowi w postaci tekstów:

```
SCHEMA.TABLE
```

Lista jest tworzona na podstawie kolumn:

```
TABLE_SCHEMA  
TABLE_NAME
```

Przykładowa pozycja może wyglądać następująco:

```
dbo.POMIARY
```

Po wybraniu pozycji tekst jest dzielony na schemat i nazwę tabeli:

```
s_schema, s_table = table_choice.split(".", 1)
```

Parametr 1 gwarantuje, że podział nastąpi tylko w pierwszym wystąpieniu kropki.

3.2 Panel serii danych

Serie są konfigurowane w panelu bocznym w sekcji:

Serie danych

Każda seria jest prezentowana w osobnym elemencie typu `expander`.

Dla każdej serii użytkownik wybiera:

- tabelę,
- zmienną,
- tryb skalowania osi Y,
- opcjonalną dolną granicę osi Y,
- opcjonalną górną granicę osi Y.

3.3 Identyfikatory serii

Każda seria posiada identyfikator generowany za pomocą biblioteki `uuid`. Identyfikator jest używany do tworzenia unikalnych kluczy widgetów Streamlit, na przykład:

```
table_<series_id>
var_<series_id>
y_auto_<series_id>
y_min_<series_id>
y_max_<series_id>
```

Dzięki temu każdy widget jest jednoznacznie przypisany do konkretnej serii.

3.4 Wybór zmiennej

Po wybraniu tabeli aplikacja pobiera listę unikalnych wartości kolumny `VARIABLE`:

```
s_variables = load_variables(
    selected_database,
    s_schema,
    s_table,
)
```

Jeżeli tabela nie zawiera żadnych zmiennych, wyświetlane jest ostrzeżenie:

Tabela nie zawiera żadnych `VARIABLE`.

Jeżeli zmienne są dostępne, użytkownik wybiera jedną z nich. Funkcja `get_variable_label` może zastąpić sam identyfikator opisową nazwą:

Nazwa opisowa (IDENTYFIKATOR)

3.5 Ręczny zakres osi Y dla serii

Każda seria posiada niezależne ustawienie zakresu osi Y. Domyślnie zaznaczona jest opcja:

Automatyczny zakres Y

W tym trybie Plotly sam dobiera zakres osi na podstawie danych. Po wyłączeniu automatycznego zakresu pojawiają się dwa pola:

Min Y
Max Y

Domyślne wartości tych pól to:

Min Y = 0.0
Max Y = 100.0

Wartości są przekazywane dalej razem z konfiguracją serii:

```
{
    "y_auto": y_auto,
    "y_min": y_min,
    "y_max": y_max,
}
```

Zakres jest stosowany tylko wtedy, gdy:

1. tryb automatyczny jest wyłączony,
2. obie wartości można skonwertować na liczby typu `float`,
3. dolna granica jest mniejsza od górnej granicy.

Jeżeli którykolwiek z tych warunków nie jest spełniony, funkcja `_get_y_range` zwraca `None`, a Plotly korzysta z automatycznego skalowania.

3.6 Dodawanie i usuwanie serii

Aktualna aplikacja pozwala na dodanie maksymalnie:

```
MAX_SERIES = 4
```

Dodanie serii odbywa się przyciskiem:

Dodaj kolejną serię

Funkcja:

```
add_series(MAX_SERIES)
```

tworzy nowy identyfikator UUID i dodaje go do listy `st.session_state.series_ids`. Jeżeli osiągnięty zostanie limit czterech serii, zamiast przycisku pojawia się informacja:

Osiągnięto maksymalną liczbę serii (4).

Usuwanie serii jest możliwe tylko wtedy, gdy istnieje więcej niż jedna seria. Po usunięciu:

- identyfikator jest usuwany z listy,
- konfiguracja serii jest usuwana,
- usuwane są wartości widgetów tabeli i zmiennej,
- aplikacja wykonuje ponowne uruchomienie.

4 Moduł config.py

Plik `config.py` zawiera wszystkie najważniejsze stałe konfiguracyjne aplikacji.

4.1 Strefa czasowa

Zdefiniowano stałą:

```
LOCAL_TZ = timezone(timedelta(hours=1))
```

Oznacza ona stałe przesunięcie UTC+1. Strefa jest wykorzystywana podczas:

- tworzenia zakresów czasu,
- obliczania czasu lokalnego,
- przeliczania timestampów pobranych z bazy,
- wyświetlania zakresu w interfejsie.

4.2 Limity i parametry wykresu

Najważniejsze parametry mają wartości:

```
MAX_PLOT_POINTS = 10_000
MAX_SERIES = 4
AXIS_GAP = 0.05
MAX_DOMAIN_START = 0.45
```

Znaczenie stałych:

- `MAX_PLOT_POINTS` – maksymalna liczba bucketów, do których agregowane są dane jednej serii,
- `MAX_SERIES` – maksymalna liczba serii na jednym wykresie,
- `AXIS_GAP` – odstęp pomiędzy dodatkowymi osiami Y,
- `MAX_DOMAIN_START` – maksymalna wartość początku obszaru wykresu na osi X.

4.3 Cache Streamlit

Aplikacja wykorzystuje dekorator `st.cache_data`, aby ograniczyć liczbę zapytań do bazy danych. Zdefiniowano następujące czasy życia cache:

```
DATA_CACHE_TTL = 300
VARIABLE_CACHE_TTL = 600
TABLE_CACHE_TTL = 600
```

Oznacza to:

- dane pomiarowe są przechowywane przez 300 sekund,
- lista zmiennych jest przechowywana przez 600 sekund,
- lista tabel i baz jest przechowywana przez 600 sekund.

Oprócz czasu życia stosowane są również limity liczby wpisów:

- `load_databases` – maksymalnie 1 wynik,
- `load_tables` – maksymalnie 8 wyników,
- `load_variables` – maksymalnie 64 wyniki,
- `load_variable_names` – maksymalnie 8 wyników,
- `load_data` – maksymalnie 64 wyniki.

Cache zależy od argumentów funkcji. Przykładowo dane pobrane dla innej tabeli, zmiennej lub zakresu czasu tworzą osobny wpis.

4.4 Konfiguracja połączenia z bazą

Połączenie jest konfigurowane za pomocą:

```
DB_SERVER = r"ZENON14WIN\ZENON_2022"
DB_DRIVER = "ODBC Driver 17 for SQL Server"
DEFAULT_DB_NAME = "Archives"
```

Tabela wykorzystywana do mapowania identyfikatorów zmiennych na nazwy opisowe to:

```
NAZWY_TABLE = "dbo.WODA_VARIABLES"
```

4.5 Paleta kolorów

Kolory serii znajdują się w liście `VARIABLE_COLORS`. Kolor jest wybierany przez funkcję:

```
def get_series_color(index: int) -> str:
    return VARIABLE_COLORS[index % len(VARIABLE_COLORS)]
```

Operator modulo powoduje, że w przypadku przekroczenia liczby dostępnych kolorów paleta jest używana cyklicznie. W aktualnej aplikacji limit czterech serii jest mniejszy niż liczba kolorów, więc każda seria otrzymuje inny kolor.

5 Moduł db.py

Plik `db.py` odpowiada za wszystkie operacje związane z bazą SQL Server. Moduł zawiera:

- konfigurację i tworzenie połączenia,
- bezpieczne składanie nazw tabel,
- pobieranie listy baz,
- pobieranie listy tabel,
- pobieranie zmiennych,
- pobieranie nazw opisowych,
- pobieranie i agregowanie danych pomiarowych.

5.1 Pooling połączeń

Na początku modułu włączany jest pooling połączeń:

```
pyodbc.pooling = True
```

Pooling pozwala na ponowne wykorzystanie wcześniej utworzonych połączeń. Ogranicza to koszt wielokrotnego tworzenia nowych połączeń przy kolejnych rerunach aplikacji Streamlit.

Nie oznacza to, że kod pozostawia otwarte połączenia. Każda funkcja otwiera połączenie, wykonuje zapytanie, a następnie zamyka połączenie w bloku `finally`.

5.2 Tworzenie połączenia

Funkcja:

```
def get_connection(database: str | None = None):
```

tworzy połączenie z SQL Serverem.

Jeżeli nie zostanie podana nazwa bazy, używana jest baza: master.

Connection string ma postać logiczną:

```
DRIVER={ODBC Driver 17 for SQL Server};  
SERVER=ZENON14WIN\ZENON_2022;  
DATABASE=Archives;  
Trusted_Connection=yes;  
Encrypt=no;  
Connection Timeout=10;
```

Parametry oznaczają:

- DRIVER – używany sterownik ODBC,
- SERVER – adres serwera SQL Server,
- DATABASE – nazwa bazy,
- Trusted_Connection=yes – uwierzytelnianie Windows,
- Encrypt=no – brak wymuszonego szyfrowania połączenia,
- Connection Timeout=10 – maksymalnie 10 sekund oczekiwania na nawiązanie połączenia.

5.3 Cytowanie nazw schematów i tabel

Funkcja:

```
def quote_identifier(value) -> str:  
    return "[" + str(value) + "]"
```

otacza nazwę schematu lub tabeli nawiasami kwadratowymi.

Nawiasy pozwalają poprawnie odwoływać się do nazw zawierających znaki specjalne lub słowa zarezerwowane przez SQL Server.

5.4 Pobieranie listy baz danych

Funkcja:

```
load_databases()
```

wykonuje zapytanie:

```
SELECT name  
FROM sys.databases  
WHERE database_id > 4  
      AND state_desc = 'ONLINE'  
ORDER BY name;
```

Warunek:

```
database_id > 4
```

pomija standardowe bazy systemowe:

- master,
- tempdb,
- model,
- msdb.

Warunek:

```
state_desc = 'ONLINE'
```

ogranicza wynik do baz, które są aktualnie dostępne.
Wynik zapytania jest odczytywany przez Pandas:

```
pd.read_sql(query, conn)["name"].tolist()
```

Ostatecznie funkcja zwraca listę tekstów zawierających nazwy baz.

5.5 Pobieranie listy tabel

Funkcja:

```
load_tables(database)
```

wykonuje zapytanie:

```
SELECT TABLE_SCHEMA, TABLE_NAME  
FROM INFORMATION_SCHEMA.TABLES  
WHERE TABLE_TYPE = 'BASE TABLE'  
ORDER BY TABLE_SCHEMA, TABLE_NAME;
```

Zapytanie zwraca wyłącznie tabele bazowe. Widoki oraz inne obiekty nie są uwzględniane.
Wynikiem jest obiekt `pandas.DataFrame` z kolumnami:

```
TABLE_SCHEMA  
TABLE_NAME
```

5.6 Pobieranie listy zmiennych

Funkcja:

```
load_variables(database, table_schema, table_name)
```

buduje pełną nazwę tabeli, a następnie wykonuje:

```
SELECT DISTINCT VARIABLE  
FROM [schema].[table]  
WHERE VARIABLE IS NOT NULL  
ORDER BY VARIABLE;
```

Słowo kluczowe `DISTINCT` powoduje usunięcie powtarzających się wartości zmiennej.
Warunek:

```
VARIABLE IS NOT NULL
```

odrzuca rekordy bez przypisanej zmiennej.
Funkcja zwraca listę wartości z kolumny `VARIABLE`.

5.7 Mapowanie identyfikatora na nazwę

Funkcja:

```
load_variable_names(database)
```

wykonuje zapytanie:

```
SELECT VARIABLE, NAME  
FROM dbo.WODA_VARIABLES;
```

Wynik jest przekształcany do słownika:

```
{  
    "101": "Temperatura",  
    "102": "Ciśnienie",  
    "103": "Wilgotność"  
}
```

Kod realizujący przekształcenie ma postać:

```
return dict(zip(df["VARIABLE"].astype(str), df["NAME"]))
```

Konwersja kolumny `VARIABLE` na tekst sprawia, że identyfikatory są porównywane w jednolity sposób niezależnie od typu zwróconego przez sterownik ODBC.

Funkcja:

```
get_variable_label(variable, names_map)
```

zwraca nazwę opisową wraz z identyfikatorem:

Temperatura (101)

Jeżeli nazwa nie istnieje w słowniku, zwracany jest sam identyfikator.

5.8 Najważniejsze zapytanie – `load_data`

Najważniejszą funkcją modułu `db.py` jest:

```
def load_data(  
    database: str,  
    table_schema: str,  
    table_name: str,  
    variable: str,  
    start_timestamp: int,  
    end_timestamp: int,  
) -> pd.DataFrame:
```

Funkcja pobiera dane dla:

- konkretnej bazy,
- konkretnego schematu,
- konkretnej tabeli,
- konkretnej wartości `VARIABLE`,
- określonego przedziału czasu.

Funkcja wykonuje również agregację danych po stronie SQL Server, dzięki czemu do Pandas i Plotly nie trafia nieograniczona liczba rekordów.

Zapytanie wykorzystuje cztery logiczne etapy zapisane jako CTE:

1. `filtered`,
2. `bounds`,
3. `bucketed`,
4. `aggregated`.

5.8.1 Etap pierwszy – CTE filtered

Pierwszy etap wybiera rekordy spełniające warunki:

```
WHERE VARIABLE = ?  
      AND TIMESTAMP_S >= ?  
      AND TIMESTAMP_S <= ?
```

Znaki zapytania są parametrami przekazywanymi do funkcji `pandas.read_sql`:

```
params=[  
    variable,  
    int(start_timestamp),  
    int(end_timestamp),  
    MAX_PLOT_POINTS,  
]
```

Parametryzacja wartości zmniejsza ryzyko błędów związanych z ręcznym wstawianiem wartości do zapytania i pozwala sterownikowi ODBC poprawnie przekazać wartości do SQL Servera.

W pierwszym CTE pobierane są również kolumny:

```
VARIABLE  
CALCULATION  
TIMESTAMP_S  
TIMESTAMP_MS  
VALUE  
STATUS  
GUID  
STRVALUE
```

Dodatkowo tworzona jest kolumna:

```
timestamp_ms_total =  
    CAST(TIMESTAMP_S AS BIGINT) * 1000  
    + COALESCE(CAST(TIMESTAMP_MS AS BIGINT), 0)
```

Kolumna `TIMESTAMP_S` przechowuje czas w sekundach, natomiast `TIMESTAMP_MS` przechowuje część milisekundową. Połączenie obu wartości w jedną liczbę milisekund umożliwia dokładne porównywanie i sortowanie rekordów. Przykładowo:

```
TIMESTAMP_S = 17000000000  
TIMESTAMP_MS = 250  
  
timestamp_ms_total = 1700000000250
```

Funkcja `COALESCE` zamienia pustą wartość milisekund na zero.

5.8.2 Etap drugi – CTE bounds

Drugim etapem jest wyznaczenie granic czasowych pobranych danych:

```
SELECT  
    MIN(timestamp_ms_total) AS min_timestamp_ms,  
    MAX(timestamp_ms_total) AS max_timestamp_ms  
FROM filtered
```

Wynik zawiera:

- najwcześniejszy timestamp,
- najpóźniejszy timestamp.

Wartości te są później wykorzystywane do podziału danych na przedziały czasowe, czyli buckety.

5.8.3 Etap trzeci – CTE bucketed

W trzecim etapie każdy rekord otrzymuje numer bucketa.
Logika ma postać:

```
CASE
  WHEN b.max_timestamp_ms = b.min_timestamp_ms THEN 0
  ELSE FLOOR(
    (f.timestamp_ms_total - b.min_timestamp_ms) * 1.0
    / NULLIF(
      b.max_timestamp_ms - b.min_timestamp_ms,
      0
    )
    * ?
  )
END AS bucket
```

Ogólny wzór można zapisać jako:

$$bucket = \left\lfloor \frac{t - t_{\min}}{t_{\max} - t_{\min}} \cdot N \right\rfloor$$

gdzie:

- t – czas danego rekordu w milisekundach,
- $t_{\min}$ – najwcześniejszy czas w pobranym zbiorze,
- $t_{\max}$ – najpóźniejszy czas w pobranym zbiorze,
- N – wartość MAX_PLOT_POINTS,
- FLOOR – zaokrąglenie w dół do liczby całkowitej.

W aktualnej konfiguracji:

$N = 10\ 000$

Jeżeli wszystkie rekordy mają identyczny timestamp, mianownik wzoru wynosi zero. Zabezpieczenie:

```
WHEN max_timestamp_ms = min_timestamp_ms THEN 0
```

umieszcza wszystkie rekordy w buckecie o numerze zero.

Funkcja NULLIF stanowi dodatkowe zabezpieczenie przed dzieleniem przez zero.

Polecenie:

```
CROSS JOIN bounds
```

udostępnia wartości minimalnego i maksymalnego czasu każdemu rekordowi z CTE **filtered**. Dzięki temu numer bucketa może zostać obliczony dla każdego wiersza.

5.8.4 Etap czwarty – CTE aggregated

W czwartym etapie rekordy są grupowane według zmiennej i numeru bucketa:

```
GROUP BY VARIABLE, bucket
```

Dla każdego bucketa wyliczane są:

```
AVG(VALUE) AS VALUE_AVG
MIN(VALUE) AS VALUE_MIN
MAX(VALUE) AS VALUE_MAX
```

Rekordy z pustą wartością są pomijane:

```
WHERE VALUE IS NOT NULL
```

Oznacza to, że:

- VALUE_AVG jest średnią wartości w buckecie,
- VALUE_MIN jest najmniejszą wartością w buckecie,
- VALUE_MAX jest największą wartością w buckecie.

Dodatkowo zachowywany jest minimalny i maksymalny timestamp w buckecie:

```
MIN(timestamp_ms_total) AS bucket_min_timestamp
MAX(timestamp_ms_total) AS bucket_max_timestamp
```

Wartości pomocnicze są wybierane za pomocą funkcji MIN:

```
MIN(CALCULATION) AS CALCULATION
MIN(STATUS) AS STATUS
MIN(GUID) AS GUID
MIN(STRVALUE) AS STRVALUE
```

W tym przypadku funkcja MIN nie oznacza wartości minimalnej w sensie pomiarowym. Służy do wybrania jednej wartości reprezentującej dany bucket.

5.8.5 Wyznaczenie czasu reprezentującego bucket

Po agregacji czas każdego bucketa jest wyznaczany jako środek przedziału:

```
(bucket_min_timestamp + bucket_max_timestamp) / 2
```

Następnie czas zostaje ponownie podzielony na sekundy i milisekundy:

```
CAST(
    (bucket_min_timestamp + bucket_max_timestamp) / 2 / 1000
    AS BIGINT
) AS TIMESTAMP_S
```

oraz:

```
CAST(
    ((bucket_min_timestamp + bucket_max_timestamp) / 2) % 1000
    AS INT
) AS TIMESTAMP_MS
```

W ten sposób każdy zagregowany punkt ma czas znajdujący się w przybliżeniu w środku zakresu czasowego, z którego powstał.

5.8.6 Sortowanie wyniku SQL

Wynik końcowy jest sortowany według:

```
ORDER BY TIMESTAMP_S ASC, TIMESTAMP_MS ASC
```

Najpierw sortowane są sekundy, a następnie milisekundy. Dzięki temu punkty na wykresie są uporządkowane chronologicznie.

5.9 Konwersja danych w Pandas

Jeżeli zapytanie nie zwróci żadnych rekordów, funkcja zwraca pusty `DataFrame` z przygotowanym zestawem kolumn:

```
VARIABLE
CALCULATION
TIMESTAMP_S
TIMESTAMP_MS
VALUE_AVG
VALUE_MIN
VALUE_MAX
STATUS
GUID
STRVALUE
time
```

Jeżeli dane istnieją, kolumny czasu są najpierw konwertowane numerycznie:

```
df["TIMESTAMP_S"] =
    pd.to_numeric(df["TIMESTAMP_S"], errors="coerce")

df["TIMESTAMP_MS"] =
    pd.to_numeric(df["TIMESTAMP_MS"], errors="coerce").fillna(0)
```

Następnie tworzona jest kolumna czasu:

```
df["time"] = pd.to_datetime(
    df["TIMESTAMP_S"],
    unit="s",
    errors="coerce",
    utc=True,
)
```

Do czasu dodawana jest część milisekundowa:

```
df["time"] = df["time"] + pd.to_timedelta(
    df["TIMESTAMP_MS"],
    unit="ms",
)
```

Na końcu czas jest konwertowany do lokalnej strefy:

```
df["time"] =
    df["time"]
    .dt.tz_convert(str(LOCAL_TZ))
    .dt.tz_localize(None)
```

W rezultacie kolumna `time` zawiera lokalny, pozbawiony informacji o strefie czasowej obiekt typu `datetime`. Kolumny wartości są konwertowane na liczby:

```
VALUE_AVG
VALUE_MIN
VALUE_MAX
```

Następnie usuwane są rekordy bez poprawnego czasu lub bez wartości AVG:

```
df = df.dropna(subset=["time", "VALUE_AVG"])
```

Ostatecznie dane są sortowane i indeks jest ustawiany od początku:

```
df = df.sort_values(
    ["time", "TIMESTAMP_MS"]
).reset_index(drop=True)
```

6 Moduł `state.py`

Plik `state.py` zawiera funkcje odpowiedzialne za stan sesji Streamlit, serie danych, zakres czasu oraz mechanizm zoomu.

6.1 Stan sesji Streamlit

Streamlit wykonuje skrypt ponownie po wielu działaniach użytkownika. Aby aplikacja nie traciła konfiguracji, dane tymczasowe są przechowywane w obiekcie:

```
st.session_state
```

Funkcja:

```
init_session_state()
```

ustawia wartości domyślne tylko wtedy, gdy dany klucz jeszcze nie istnieje.

6.2 Domyślna seria

Jeżeli lista serii nie istnieje, tworzona jest jedna seria z identyfikatorem UUID:

```
st.session_state.series_ids = [str(uuid.uuid4())]
```

Dzięki temu aplikacja po uruchomieniu zawsze posiada miejsce na pierwszą konfigurację. Konfiguracje serii są przechowywane w:

```
st.session_state.series_data
```

6.3 Domyślny zakres czasu

Przy pierwszym uruchomieniu zakres jest ustawiany na ostatnią godzinę:

```
now = datetime.now(LOCAL_TZ)

time_start_original = now - timedelta(hours=1)
time_end_original = now
```

Przechowywane są dwa rodzaje zakresu:

- `time_start_original` i `time_end_original` – zapamiętany zakres sprzed zoomu,
- `time_start` i `time_end` – zakres aktualnie używany do pobierania danych.

6.4 Tryby wyboru zakresu

Aplikacja obsługuje trzy tryby:

```
"Ostatnie ciągle"
"Ostatnie dyskretne"
"Własny zakres"
```

Domyślnie ustawiany jest tryb:

```
"Ostatnie dyskretne"
```

oraz zakres:

```
"1 godzina"
```

6.5 Szybkie zakresy czasu

Zdefiniowano słownik:

```
QUICK_RANGES = {
    "10 minut": timedelta(minutes=10),
    "1 godzina": timedelta(hours=1),
    "1 dzień": timedelta(days=1),
    "1 tydzień": timedelta(weeks=1),
    "1 miesiąc": timedelta(days=30),
    "1 rok": timedelta(days=365),
}
```

Tryb dyskretny pozwala wybrać jeden z gotowych przedziałów. Początek zakresu jest obliczany jako:

```
start_time = now - QUICK_RANGES[quick_range]
end_time = now
```

6.6 Tryb ostatniego zakresu ciągłego

W trybie ciągłym użytkownik podaje niezależnie liczbę:

- sekund,
- minut,
- godzin,
- dni.

Następnie wartości są łączone w obiekt `timedelta`:

```
delta = timedelta(
    seconds=range_seconds,
    minutes=range_minutes,
    hours=range_hours,
    days=range_days,
)
```

Zakres jest obliczany od aktualnego czasu:

```
start_time = now - delta
end_time = now
```

6.7 Własny zakres daty i godziny

W trybie własnym użytkownik wybiera:

- datę początkową,
- godzinę początkową,
- datę końcową,
- godzinę końcową.

Następnie data i czas są łączone:

```
start_time = datetime.combine(  
    start_date,  
    start_clock,  
) .replace(tzinfo=LOCAL_TZ)
```

Analogicznie tworzony jest czas końcowy.

Jeżeli początek zakresu jest późniejszy od końca, aplikacja wyświetla błąd:

Data/godzina początkowa musi być wcześniejsza niż końcowa.

6.8 Funkcja insert_time

Funkcja:

```
insert_time(start_time, end_time)
```

zapamiętuje zakres czasu w odpowiednich widgetach.

Działanie zależy od bieżącego trybu.

W trybie dyskretnym funkcja oblicza różnicę pomiędzy aktualnym czasem a początkiem zakresu, a następnie wybiera najbliższy element z `QUICK_RANGES`.

W trybie ciągłym różnica jest rozbijana na:

- dni,
- godziny,
- minuty,
- sekundy.

W trybie własnym aktualizowane są pola dat i godzin.

Niezależnie od trybu funkcja zawsze aktualizuje:

```
time_start_original  
time_end_original  
time_start  
time_end
```

6.9 Dodawanie serii

Funkcja:

```
add_series(max_series)
```

sprawdza, czy liczba serii jest mniejsza od limitu.

Jeżeli aplikacja nie jest w trybie zoomu, funkcja tymczasowo zapamiętuje aktualny tryb zakresu, wywołuje `insert_time`, a następnie przywraca tryb. Zapobiega to utracie ustawień zakresu po rerunie Streamlit.

Następnie generowany jest nowy UUID:

```
str(uuid.uuid4())
```

i dodawany do listy serii.

6.10 Usuwanie serii

Funkcja:

```
remove_series(series_id)
```

usuwa wskazaną serię.

Usuwane są:

- identyfikator z `series_ids`,
- konfiguracja z `series_data`,
- klucz widgetu tabeli,
- klucz widgetu zmiennej.

Jeżeli aplikacja nie jest w trybie zoomu, przed usunięciem serii zachowywany jest aktualny zakres czasu.

6.11 Resetowanie serii

Funkcja:

```
reset_series()
```

jest wywoływana po zmianie bazy danych.

Dla wszystkich dotychczasowych serii usuwane są zapisane konfiguracje i klucze widgetów. Następnie tworzona jest jedna nowa seria.

Reset nie usuwa zakresu czasu ani ustawień trybu wyboru czasu.

6.12 Mechanizm box-select i zoomu

Wykres Plotly może wysłać do aplikacji zdarzenie zaznaczenia prostokątnego. Zdarzenie to jest obsługiwane przez funkcję:

```
handle_zoom_box_select(box_x)
```

Funkcja oczekuje listy zawierającej co najmniej dwa punkty czasu.

Najpierw tworzony jest podpis zaznaczenia:

```
box_signature = tuple(box_x)
```

Jeżeli identyczne zaznaczenie było już wcześniej przetworzone, funkcja zwraca `False`. Mechanizm ten zapobiega wielokrotnemu przetwarzaniu tego samego zdarzenia podczas kolejnych rerunów.

Następnie sprawdzana jest kolejność punktów. Jeżeli pierwszy punkt jest późniejszy od drugiego, wartości są zamieniane:

```
if new_start > new_end:  
    new_start, new_end = new_end, new_start
```

Przy pierwszym wejściu w tryb zoomu zapisywany jest poprzedni tryb zakresu:

```
pre_zoom_range_mode = range_mode
```

Nowy zakres jest zapisywany jako:

```
time_start = new_start.replace(tzinfo=LOCAL_TZ)  
time_end = new_end.replace(tzinfo=LOCAL_TZ)
```

oraz ustawiana jest flaga:

```
zoom_active = True
```

W przeciwieństwie do zwykłego zoomu Plotly, box-select zmienia zakres pobierania danych. Oznacza to, że po zaznaczeniu fragmentu dane są ponownie pobierane z bazy, tym razem dla krótszego zakresu i z większą szczegółowością.

6.13 Cofnięcie zoomu

Funkcja:

```
undo_zoom()
```

przywraca zakres obowiązujący przed wejściem w tryb zoomu.

Najpierw odtwarzany jest poprzedni tryb:

```
restore_mode = pre_zoom_range_mode or range_mode
```

Następnie funkcja wywołuje:

```
insert_time(
    time_start_original,
    time_end_original,
)
```

Po przywróceniu:

```
zoom_active = False
last_processed_box = None
pre_zoom_range_mode = None
```

Zwiększany jest również:

```
chart_key_version
```

Dzięki temu Streamlit może potraktować wykres jako nową wersję komponentu.

7 Moduł charting.py

Plik `charting.py` zawiera logikę tworzenia wykresu Plotly oraz renderowania fragmentu interfejsu odpowiedzialnego za wizualizację.

Moduł obsługuje:

- osie Y dla wielu serii,
- automatyczne i ręczne zakresy osi,
- wartości AVG, MIN i MAX,
- markery M1 i M2,
- zdarzenia kliknięcia punktów,
- box-select,
- tabelę podsumowania,
- tabelę informacji o markerach,
- automatyczne odświeżanie fragmentu wykresu.

7.1 Unikalne klucze wykresu

Funkcja:

```
_widget_key(chart_key, name)
```

tworzy unikalny klucz:

```
f"{chart_key}_{name}"
```

Funkcja:

```
_chart_keys(chart_key)
```

tworzy zestaw kluczy dla:

- opcji MIN/MAX,
- wszystkich osi po lewej,
- markerów,
- pierwszego i drugiego markera,
- podpisu ostatniego kliknięcia,
- komponentu wykresu,
- przycisku czyszczenia markerów.

Takie rozwiązanie pozwala bezpiecznie używać więcej niż jednego wykresu w tej samej aplikacji, o ile każdy otrzyma odmienny `chart_key`.

7.2 Konwersja czasu

Funkcja:

```
_to_naive_timestamp(value)
```

konwertuje wartość do `pandas.Timestamp` bez informacji o strefie czasowej.
Jeżeli wartość jest pusta lub niepoprawna, zwracane jest `None`.

Funkcja:

```
_to_python_datetime(value)
```

konwertuje timestamp do standardowego obiektu `datetime.datetime`.

Funkcja:

```
_format_dt(value)
```

formatuje czas do postaci:

```
YYYY-MM-DD HH:MM:SS.mmm
```

Milisekundy są ograniczane do trzech cyfr.

7.3 Konfiguracja osi Y

Funkcja:

```
_get_y_range(series)
```

odczytuje ustawienia osi Y zapisane w konfiguracji serii.
Jeżeli:

```
series["y_auto"] == True
```

zwracane jest:

`None`

co oznacza automatyczne skalowanie.

W trybie ręcznym wartości są konwertowane:

```
y_min = float(series.get("y_min"))
y_max = float(series.get("y_max"))
```

Zakres jest zwracany wyłącznie wtedy, gdy:

$$y_{\min} < y_{\max}$$

W przeciwnym przypadku oś pozostaje w trybie automatycznym.

7.4 Rozmieszczanie osi Y

Funkcja:

```
_axis_for_series(
    series_idx,
    label,
    color,
    y_range,
    all_axes_left,
    domain_start,
    extra_used,
)
```

tworzy konfigurację osi dla konkretnej serii.
Dla pierwszej serii używana jest główna oś:

y

i jest ona umieszczana po lewej stronie.

Jeżeli opcja wszystkich osi po lewej jest wyłączona, druga seria otrzymuje:

`y2`

po prawej stronie. Oś ta nakłada się na główną oś Y poprzez:

```
overlying="y"  
side="right"
```

Kolejne serie otrzymują dodatkowe osie po lewej stronie. Są one zakotwiczone swobodnie:

```
anchor="free"
```

i otrzymują pozycję obliczaną na podstawie:

```
position = max(  
    0.0,  
    domain_start - AXIS_GAP * extra_used  
)
```

Jeżeli opcja:

`Wszytkie osie po lewej`

jest włączona, wszystkie serie poza pierwszą otrzymują dodatkowe osie po lewej stronie.

7.5 Dodawanie wykresów Plotly

Funkcja:

```
_add_series_trace(...)
```

dodaje do figure obiekt:

```
go.Scatter(...)
```

Wykres może być tworzony jako:

```
mode="lines"
```

lub:

```
mode="lines+markers"
```

W aktualnej implementacji główna linia AVG jest tworzona z markerami, natomiast ślady MIN i MAX są tworzone jako linie kropkowane bez markerów.

Dla każdego śladu ustawiane są między innymi:

- kolor,
- szerokość linii,
- styl linii,
- oś Y,
- dane pomocnicze `customdata`,
- sposób wyświetlania legendy,
- format odpowiedzi po najechaniu.

Przykładowy format odpowiedzi:

Czas: 2026-09-04 12:00:00

AVG: 12.345678

7.6 Budowa figury

Główna funkcja tworząca wykres to:

```
_build_figure(  
    all_series,  
    start_time,  
    end_time,  
    show_minmax,  
    all_axes_left,  
    markers_enabled=False,  
)
```

Funkcja oblicza liczbę dodatkowych osi:

```
extra_left = max(  
    0,  
    num_series - (1 if all_axes_left else 2),  
)
```

Następnie ustala początek obszaru osi X:

```
domain_start = min(  
    extra_left * AXIS_GAP,  
    MAX_DOMAIN_START,  
)
```

Dla każdej serii sprawdzana jest obecność kolumn:

```
time  
VALUE_AVG  
VALUE_MIN  
VALUE_MAX
```

Serie bez kompletnych danych są pomijane.

Dla każdej poprawnej serii:

- tworzona jest odpowiednia oś Y,
- opcjonalnie dodawana jest linia MIN,
- opcjonalnie dodawana jest linia MAX,
- dodawana jest główna linia AVG.

7.7 Wartości MIN i MAX

Jeżeli użytkownik zaznaczy:

Pokaż zakres MIN/MAX

dla każdej serii dodawane są dwa dodatkowe ślady:

```
MIN - nazwa serii  
MAX - nazwa serii
```

7.8 Układ wykresu

Wykres jest tworzony z następującymi parametrami:

```
height=700  
template="plotly_dark"  
hovermode="closest"  
clickmode="event+select"  
dragmode="zoom"
```

Oznacza to:

- wysokość wykresu wynosi 700 pikseli,
- używany jest ciemny szablon Plotly,

- odpowiedź dotyczy najbliższego punktu,
- wykres obsługuje zdarzenia kliknięcia i zaznaczenia,
- domyślnym trybem przeciągania jest zwykły zoom.

Legenda ma orientację poziomą i jest umieszczona poniżej wykresu.
Oś X ma tytuł:

Czas

oraz zakres wynikający z `start_time` i `end_time`.

7.9 Obsługa zdarzeń Plotly

7.9.1 Odczyt atrybutów zdarzenia

Funkcja:

```
_get_attr(obj, name, default=None)
```

pozwała odczytywać atrybuty zarówno ze zwykłego słownika, jak i z obiektu zwracanego przez Streamlit lub Plotly.
Jeżeli obiekt jest słownikiem, używany jest:

```
obj.get(name, default)
```

W przeciwnym przypadku używane jest:

```
getattr(obj, name, default)
```

Takie rozwiązanie zwiększa odporność kodu na różne reprezentacje obiektu zdarzenia.

7.9.2 Punkty i zaznaczenia box-select

Funkcje:

```
_get_selection(chart_event)  
_get_selection_points(chart_event)  
_get_box_selections(chart_event)
```

wydobywają z obiektu zdarzenia:

- sekcję `selection`,
- kliknięte punkty,
- prostokątne zaznaczenia box-select.

Jeżeli brak jest odpowiednich danych, zwracane są puste słowniki lub listy.

7.9.3 Rozpoznawanie serii na podstawie customdata

Każdy ślad otrzymuje:

```
customdata = [series_idx] * len(data)
```

Dzięki temu po kliknięciu punktu można określić, do której serii należy.

Funkcja:

```
_extract_series_idx(customdata)
```

konwertuje wartość na indeks serii.

W kodzie znajduje się również awaryjne mapowanie numeru śladu na serię:

```
_series_index_from_curve(curve_number, show_minmax)
```

Jeżeli wyświetlane są MIN i MAX, jedna seria może mieć trzy ślady:

1. MIN,
2. MAX,
3. AVG.

W przeciwnym przypadku jedna seria ma jeden główny ślad.

7.10 Markery M1 i M2

7.10.1 Włączenie trybu markerów

Tryb markerów jest aktywowany checkboxem:

Markery klikane

Po włączeniu aplikacja interpretuje kliknięcia punktów wykresu jako wybór markerów. Jeżeli tryb markerów jest wyłączony, użytkownik otrzymuje informację:

Przeciagnij po wykresie, aby zrobić zaznaczenie (Box Select) i przybliżyć zakres.

Jeżeli tryb markerów jest włączony, wyświetlana jest instrukcja:

Kliknij bezpośrednio punkt na wykresie.
Pierwszy klik=M1, drugi=M2, trzeci przesuw M1/M2.

7.10.2 Odczyt klikniętego punktu

Funkcja:

```
_get_last_clicked_point(chart_event)
```

pobiera ostatni punkt z listy zaznaczonych punktów.
Dla punktu zapisywane są:

- współrzędna czasu x,
- wartość y,
- numer śladu,
- numer punktu,
- indeks punktu,
- indeks serii.

Przykładowa struktura markera:

```
{
  "x": czas,
  "y": wartość,
  "curve_number": 2,
  "point_number": 15,
  "point_index": 15,
  "series_idx": 0,
}
```

7.10.3 Zapisywanie dwóch markerów

Markery są przechowywane jako:

marker_1
marker_2

Pierwszy klik zapisuje marker M1.
Drugi klik zapisuje marker M2.
Trzeci klik powoduje przesunięcie:

stary M2 staje się nowym M1
nowy punkt staje się M2

Mechanizm ten pozwala szybko analizować kolejne pary punktów bez ręcznego czyszczenia markerów. Aby uniknąć wielokrotnego przetwarzania tego samego zdarzenia, zapisywany jest podpis:

last_marker_signature

7.10.4 Wizualizacja markerów

Funkcja:

```
_add_marker_traces(fig, all_series, show_minmax, keys)
```

dodaje markery do figury jako osobne ślady Plotly.

Marker M1 ma kolor: 00e5ff

Marker M2 ma kolor: 32CD32

Markery są tworzone jako:

```
mode="markers+text"
```

Dodatkowo dla każdego markera rysowane są:

- pionowa linia pomocnicza przez czas markera,
- pozioma linia pomocnicza przez wartość markera.

7.10.5 Informacje o markerach

Po ustawieniu markerów aplikacja wyświetla tabelę zawierającą:

- serię,
- czas M1,
- wartość M1,
- czas M2,
- wartość M2,
- różnicę wartości ΔY ,
- różnicę czasu Δt w sekundach,
- tempo zmiany $\Delta Y / \Delta t$.

Tabela jest wyświetlana tylko dla serii, na których ustawiono przynajmniej jeden marker.

7.10.6 Czyszczenie markerów

Przycisk:

Wyczyść markery

ustawia:

```
marker_1 = None
marker_2 = None
last_marker_signature = None
```

Po wyczyszczeniu fragment wykresu jest ponownie renderowany.

8 Fragment Streamlit i automatyczne odświeżanie

8.1 Funkcja make_chart_fragment

Funkcja:

```
make_chart_fragment(
    refresh_seconds,
    chart_key="main_chart",
)
```

tworzy funkcję opakowaną dekoratorem:

```
@st.fragment(run_every=refresh_seconds)
```

Fragment pozwala odświeżać część aplikacji odpowiedzialną za wykres, bez konieczności pełnego przebudowywania całego interfejsu.

Zwracana funkcja przyjmuje:

```
all_series
start_time
end_time
```

i przekazuje je do:

```
_render_chart_body(...)
```

8.2 Ustawienia odświeżania

W panelu bocznym użytkownik może wybrać:

```
Wyłączone
1 minuta
5 minut
15 minut
30 minut
```

Wartości są mapowane na sekundy:

```
"Wyłączone" = None
"1 minuta" = 60
"5 minut" = 300
"15 minut" = 900
"30 minut" = 1800
```

Wartość `None` wyłącza automatyczne odświeżanie fragmentu.

8.3 Ręczne odświeżenie

Przycisk:

Odśwież teraz

wykonuje:

```
load_data.clear()
st.rerun()
```

Oznacza to, że cache danych pomiarowych zostaje wyczyszczony, a aplikacja pobiera dane ponownie z bazy SQL Server. Pozostałe cache, takie jak lista tabel lub zmiennych, nie są w tym miejscu czyszczone.

9 Update

9.1 Wielopoziomowe cofanie zoomu (state.py)

Dodano stos `zoom_history`, który przechowuje wyłącznie zakresy pośrednie. Zakres bazowy nadal jest trzymany w `time_start_original` i `time_end_original`.

- `init_session_state`: inicjalizacja `zoom_history` = [].
- `handle_zoom_box_select`: gdy `zoom_active` jest już prawdziwe, bieżący zakres jest odkładany na stos przed ustawieniem nowego; przy pierwszym zoomie stos jest czyszczony, a `range_mode` zapamiętywany w `pre_zoom_range_mode`.
- `undo_zoom`: jeśli stos nie jest pusty, zdejmuje jeden poziom i przywraca go do `time_start` i `time_end`; gdy stos jest pusty, wykonuje pełne przywrócenie (`range_mode`, zakres bazowy, `zoom_active` = `False`).

9.2 Prawy przycisk myszy = box select, lewy = zoom (charting)

Lewy przycisk wykonuje zwykły zoom Plotly po stronie przeglądarki (bez pobierania danych). Prawy przycisk wykonuje box select, który przechodzi przez dotychczasowy mechanizm: `on_select="rerun"` zwraca zaznaczenie do Pythona, a `handle_zoom_box_select` ustawia nowy zakres i dane są pobierane na nowo.

Rozwiązanie to skrypt JavaScript wstrzykiwany przez `components.html` do strony nadrzędnej:

- skrypt jest dodawany jako tag `<script>` do `window.parent.document`, jednorazowo (flaga `window.__box`), więc przeżywa odświeżanie iframe'a komponentu;
- zdarzenie `contextmenu` nad wykresem jest blokowane;
- prawy `mousedown` nad warstwą `.draglayer` jest przechwytywany (faza capture), a tryb przeciągania jest przełączany na `select` przez przycisk „Box Select” z paska Plotly;
- po krótkiej zwłoce generowane jest sztuczne zdarzenie `mousedown` z `button: 0`, a kolejne ruchy myszy są przekazywane jako `mousemove` z lewym przyciskiem;
- po puszczeniu prawego przycisku wysyłany jest `mouseup`, a tryb wraca do `zoom`;
- wywołanie `_install_right_drag_box_select()` znajduje się na początku `_render_chart_body`.

9.3 Naprawa znikających linii MIN/MAX po cofnięciu

Zmienna `chart_key_version` nie była nigdzie odczytywana w kodzie wykresu, a `uirevision` był stały. Wprowadzono trzy zmiany:

1. klucz wykresu w `_chart_keys` zależy od wersji;
2. `uirevision` w `_build_figure` zależy od wersji;
3. `chart_key_version` jest zwiększany także w `handle_zoom_box_select`.

Dzięki temu cofnięcie i nowy box select resetują stan wykresu po stronie przeglądarki, a zwykły zoom lewym przyciskiem pozostaje zachowany między automatycznymi odświeżeniami.